# EllipseNet: YOLO-Style Oriented Ellipse Detection

Applied Machine Learning Final Project | Spring 2026

Hailemariam | NetID: hbm9834

Goal: modify a YOLO-style detector to predict oriented ellipses instead of rectangular boxes using Pascal VOC. The implementation includes a NumPy-from-scratch detector and an advanced pretrained ResNet-18 backbone version.

## Project requirements covered

- Dataset customization: VOC boxes are converted into ellipse annotations.
- Network output modification: each cell predicts objectness, ellipse geometry, angle, and class logits.
- Rotation augmentation: image rotation updates ellipse center and theta targets.
- Ellipse IoU and NMS: overlap is computed for filtering and mAP evaluation.
- Loss integration: localization, objectness, no-objectness, class, and angle terms.
- Hyperparameter search: learning rate, thresholds, and selected loss weights.
- Metrics and visualizations: mAP, precision, recall, loss curves, predictions, and overfitting check.

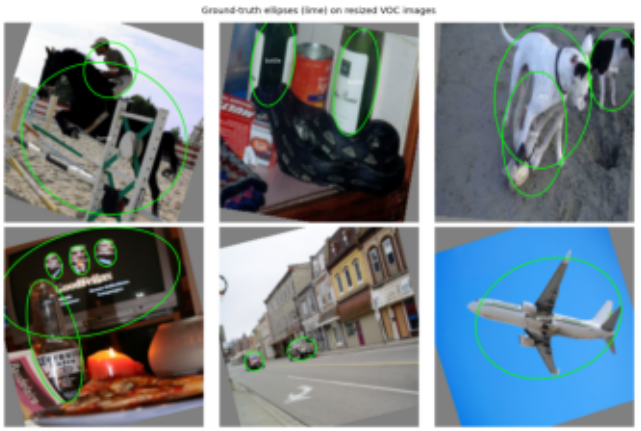
# Dataset and Mathematical Target

For each VOC bounding box, an ellipse is inscribed inside the box. All coordinates are normalized by image width W and height H.

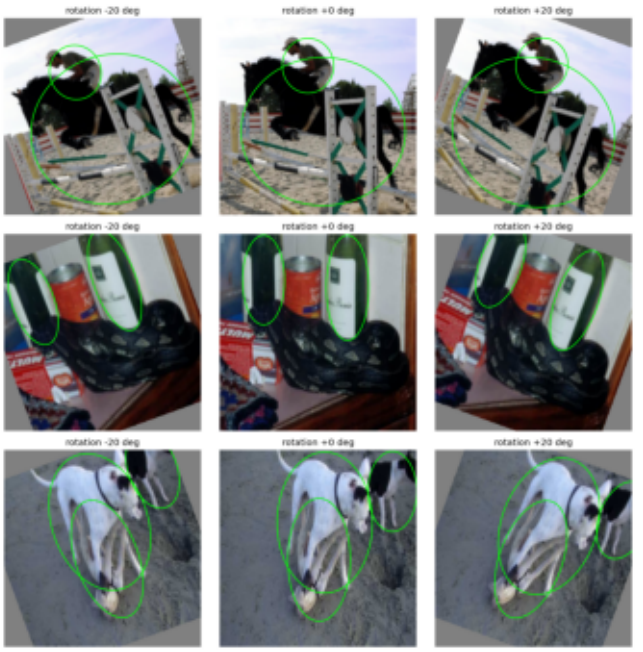
```
cx = (xmin + xmax) / (2W)        cy = (ymin + ymax) / (2H)
a  = (xmax - xmin) / (2W)        b  = (ymax - ymin) / (2H)
theta = 0 for original VOC boxes
```

```
For a rotation alpha around image center (0.5, 0.5):
[cx', cy'] = rotate([cx, cy], alpha)
theta' = (theta + alpha) mod pi
```

**Ground-truth ellipse overlays**




**Rotation augmentation preview**



# Architecture and Output Design

Final NumPy model: input 224x224, grid S=7, B=2 ellipse predictors per cell, C=20 VOC classes. Output tensor per image is 7 x 7 x 2 x 26.

```
Per predictor output:
[o, tx, ty, ta, tb, ttheta, class logits x 20]

Decoded ellipse:
cx_hat = (sigmoid(tx) + grid_x) / S
cy_hat = (sigmoid(ty) + grid_y) / S
a_hat  = anchor_a * exp(ta)
b_hat  = anchor_b * exp(tb)
theta_hat = pi * sigmoid(ttheta)
score = sigmoid(o) * max(softmax(class_logits))
```

Backbone: small CPU-friendly NumPy CNN with 3x3 convolutions, 1x1 bottleneck convolutions, max pooling, fully connected layer, and detection head. Downsampling path is 224 -> 112 -> 56 -> 28 -> 14 -> 7. Total scalar parameters: 2,992,188.

| Component | Specification |
|---|---|
| Input | 3 x 224 x 224 |
| Grid | 7 x 7 |
| Predictors | B = 2 per cell |
| Classes | 20 VOC classes |
| Output | 7 x 7 x 2 x 26 |
| Parameters | 2,992,188 |

# Loss Function and Ellipse IoU

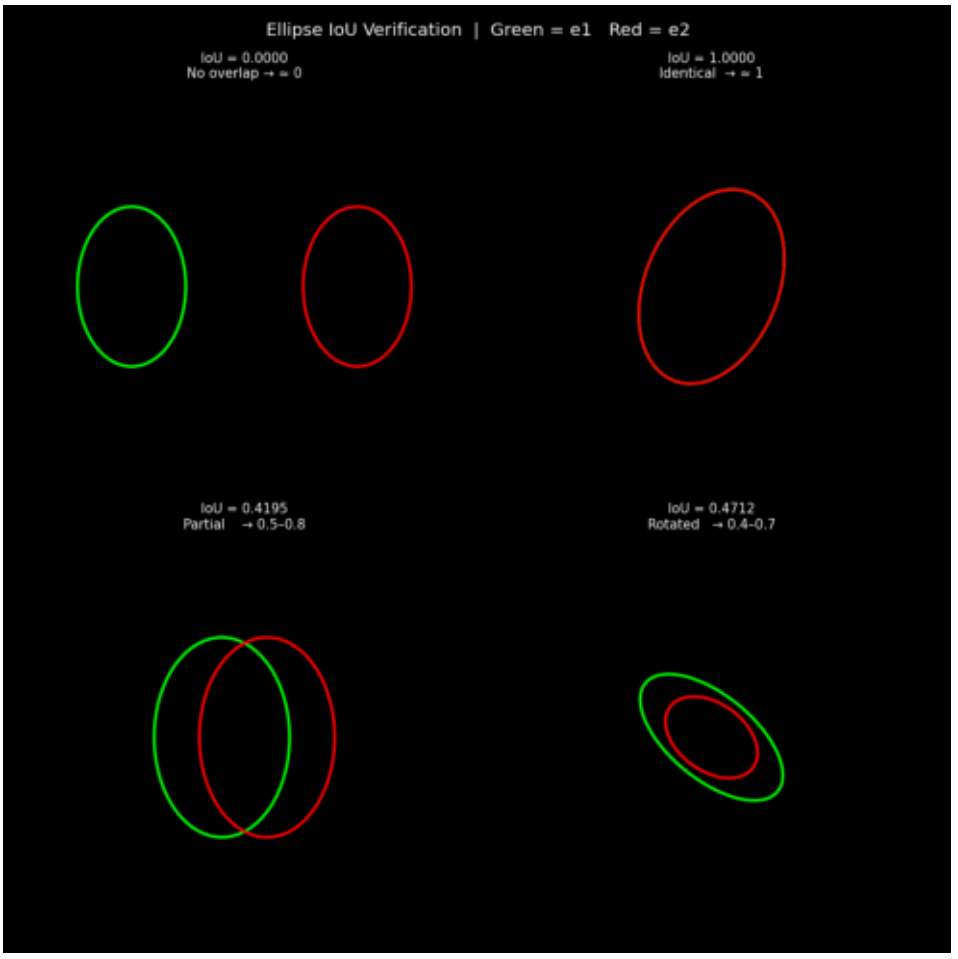
```
Total loss:
L = lambda_loc L_loc + lambda_obj L_obj + lambda_noobj L_noobj
    + lambda_cls L_cls + lambda_theta L_theta

L_loc = MSE(center offsets) + MSE(log axes) + MSE(sqrt axes)
L_obj, L_noobj = binary cross entropy
L_cls = softmax cross entropy
L_theta = min(|theta_hat - theta|, pi - |theta_hat - theta|)

Ellipse IoU is estimated by Monte Carlo sampling:
IoU(E1,E2) ~= count(points in E1 and E2) / count(points in E1 or E2)
```

Design decision: ellipse IoU is used for NMS and evaluation. The training objective uses differentiable coordinate losses because the NumPy model implements manual backpropagation.

**Ellipse IoU verification**



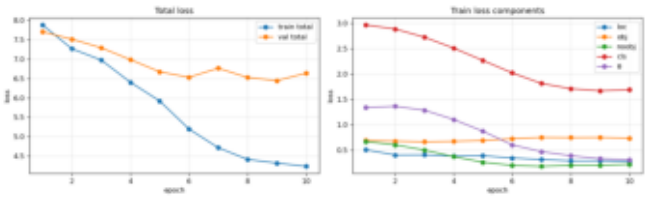
# Experiment 1: Full From-Scratch Training

Full training used 5,717 VOC train samples and 5,823 validation samples. The best validation checkpoint occurred at epoch 9.

| Run point | Train loss | Val loss | Note |
|---|---|---|---|
| Epoch 1 | 5.7543 | 5.2136 | initial |
| Best ckpt | 3.5350 | 4.6092 | epoch 9 |
| Epoch 30 | 1.4056 | 5.2631 | overfitting |

| Split | mAP | Precision | Recall |
|---|---|---|---|
| Train | 0.0859 | 0.1462 | 0.1333 |
| Validation | 0.0560 | 0.0960 | 0.0996 |

**Loss curves**



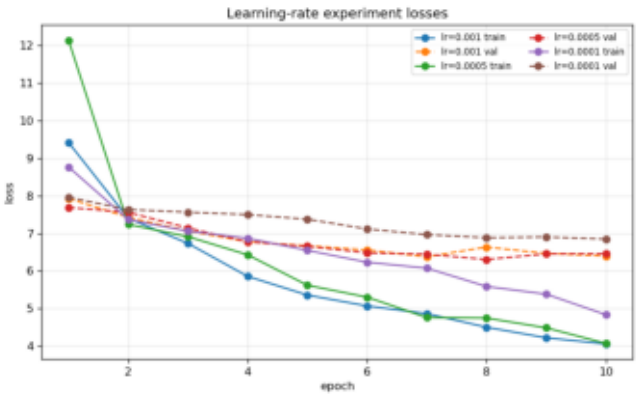
**Sample predictions**

# Experiment 2: Hyperparameter Search

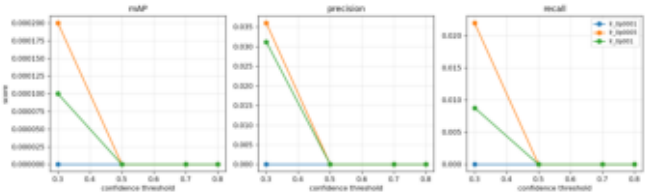
Learning-rate and confidence-threshold tests were run on a staged dataset. The best validation loss among tested learning rates was obtained with LR = 5e-4.

| Learning rate | Best val loss | Best conf. | Best mAP |
|---|---|---|---|
| 1e-3 | 6.3755 | 0.3 | 0.0001 |
| 5e-4 | 6.3055 | 0.3 | 0.0002 |
| 1e-4 | 6.8470 | 0.3 | 0.0000 |

**LR experiment losses**




**Threshold sweep summary**

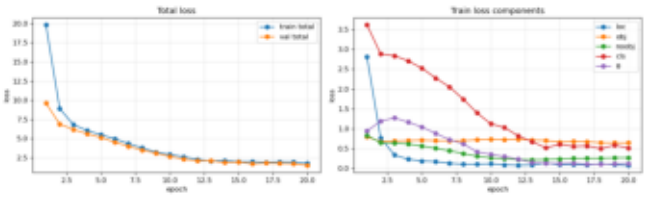


# Experiment 3: Overfitting Check

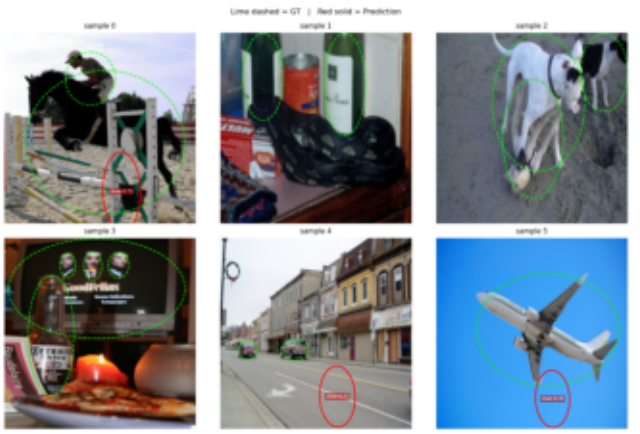
A deterministic 20-image memorization run verified that the pipeline can learn when the task is small. This checks target assignment, gradient flow, decoding, and visualization.

| Epoch | Train loss | Val loss |
|---|---|---|
| 1 | 19.8149 | 9.6162 |
| 20 | 1.7805 | 1.4900 |

**Overfitting loss**




**Overfitting predictions**



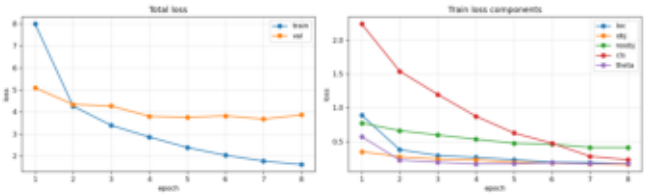
# Experiment 4: Pretrained Backbone

An advanced version adds an ImageNet-pretrained ResNet-18 backbone. The original classifier is removed, an ellipse detection head is attached, the backbone is frozen for the first epochs, and then all layers are fine-tuned. The updated pretrained notebook supports 448x448 input with adaptive pooling to keep a 7x7 grid.

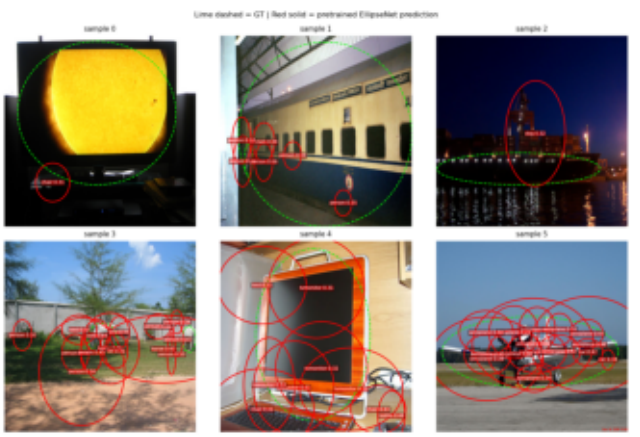
| Confidence | mAP | Precision | Recall |
|---|---|---|---|
| 0.30 | 0.0617 | 0.0528 | 0.3297 |
| 0.50 | 0.0538 | 0.1036 | 0.2817 |
| 0.70 | 0.0409 | 0.1738 | 0.2227 |

Result: the pretrained staged model produced higher recall than the NumPy-from-scratch model, showing that pretrained visual features are useful for this detection task.

**Pretrained loss curves**



**Pretrained predictions**



# Conclusion

- The final detector satisfies the project requirements: customized ellipse annotations, modified output head, IoU/NMS, integrated loss, hyperparameter search, mAP metrics, and visualizations.
- The NumPy model trains correctly and can overfit a small dataset, confirming that the training and prediction pipeline works.
- Full VOC performance is limited by training from scratch, small model capacity, confidence calibration, and ellipse targets derived from rectangular boxes.
- The pretrained ResNet-18 version improves feature quality and recall, making it the strongest direction for future performance improvements.